

Evaluation of LP Relaxation Solver Strategies in CBC

Analysis of `guess()` Parameter Configurations

CBC Development

March 2026

Contents

1	Overview	2
2	Experimental Setup	2
2.1	Benchmark Set	2
2.2	Solver Configurations	2
2.3	Protocol	2
3	Correctness and Reliability	2
3.1	Memory Safety	2
3.2	Per-Method Problem Summary	3
3.3	False Objective Matches	3
3.4	Instances with No Solution (NA / Killed)	3
4	Performance Analysis	4
4.1	Summary Statistics	4
4.2	Fastest-Method Count and Rank Distribution	4
4.3	Performance Profile	4
4.4	Geometric Mean Ratio vs Virtual Best	5
4.5	Pairwise Comparison	5
4.6	Significant Wins (Method-Specific Advantage)	5
4.7	Extreme Cases	6
5	Conclusions and Recommendations	6
5.1	Memory Safety	6
5.2	Correctness	6
5.3	Reliability	6
5.4	Performance	6
5.5	Recommended Action	7

1 Overview

The CBC solver’s `-guess` option (implemented in `ClpSimplexOther::guess()`) selects one of four LP relaxation configurations based on instance characteristics (integrality of variables and statistics of objective coefficients). The goal of this experiment is to evaluate each configuration individually on a representative benchmark set in order to:

- identify correctness problems (wrong objective, primal infeasibilities);
- identify reliability problems (time-limit kills, no solution produced);
- determine which configurations have a genuine performance advantage over the others, and whether any can be removed from the selection pool.

All runs used the ASan+debug build (`-O1 -g -fsanitize=address`) installed at `~/prog/cbc-dbg/bin/cbc`, to additionally detect memory errors and assertion failures.

2 Experimental Setup

2.1 Benchmark Set

The benchmark consists of **1 317 MIP instances** from the `~/inst/super/` collection. Reference LP relaxation objectives were obtained from Gurobi (`lp_relaxation_results_gurobi.tsv`).

2.2 Solver Configurations

The four configurations correspond exactly to the branches inside `guess()` and are selected by two binary conditions: whether all variables with $\ell < u$ are integer (`allInt`), and whether the median or average absolute objective coefficient exceeds a threshold.

Table 1: The four LP solver configurations tested.

Name	Condition	CBC options
idiot30	<code>allInt</code> , $\text{avg} \leq 0.0086$	<code>-idiot 30 -pertvalue -1483 -primals</code>
idiot60	<code>allInt</code> , $\text{avg} > 0.0086$	<code>-idiot 60 -primals</code>
dualpesteepest	<code>!allInt</code> , $\text{median} \leq 0.75$	<code>-dualpivot pesteepest -psi 1.0 -pertv 52 -duals</code>
idiot80	<code>!allInt</code> , $\text{median} > 0.75$	<code>-idiot 80 -primals</code>

2.3 Protocol

Every instance was solved by *all four* configurations independently (i.e., ignoring the actual `guess()` decision), to enable direct head-to-head comparison. Per-instance time limits were taken from `suggested_lp_time_limits.tsv` (formula: $\min(7200, \max(2t_{\text{ref}}, t_{\text{ref}} + 1800))$) where t_{ref} is the `initialSolve` time on the same instance). Jobs were run in parallel with 120 cores using GNU parallel. Total jobs: $1317 \times 4 = 5268$.

3 Correctness and Reliability

3.1 Memory Safety

Zero ASan errors were detected across all 5 268 runs. No assertion failures were observed either. All four configurations are memory-safe on this benchmark set.

3.2 Per-Method Problem Summary

Table 2 summarises correctness and reliability issues per method. A “problem” is any of: killed by the time limit (no solution), objective not parseable (NA), or objective value differing from the Gurobi reference by more than a relative tolerance of 10^{-6} .

Table 2: Correctness and reliability per configuration (out of 1317 instances).

Method	Solved	Killed/NA	False	Total problems	Problem rate
idiot30	1276	40	1	41	3.1%
idiot60	1284	32	1	33	2.5%
idiot80	1283	33	1	34	2.6%
dualpesteeep	1285	30	2	32	2.4%

Key observations:

- **idiot30** has the highest problem rate, driven by the most time-limit kills (40). The idiot algorithm with only 30 iterations is the least reliable on harder instances.
- **dualpesteeep** has the fewest kills (30) but the most false objective matches (2), indicating an accuracy issue in specific cases (see Section 3.3).
- **idiot60** and **idiot80** are nearly identical in reliability.

3.3 False Objective Matches

Five runs produced an objective value that does not match the Gurobi reference. All involve instances where CBC reports **Unscaled problem has primal infeasibilities** after declaring optimality, indicating a post-solve scaling issue that corrupts the reported objective.

Table 3: False objective matches (reference value from Gurobi).

Instance	Method	CBC obj	Gurobi ref	Log message
gsvm2r15	dualpesteeep	0.19932	0.20000	primal infeasibilities
irish-electricity	dualpesteeep	2 454 717	2 546 255	primal infeas. + barrier
irish-electricity	idiot30	2 454 743	2 546 255	primal infeas. + barrier
irish-electricity	idiot60	2 454 759	2 546 255	primal infeas. + barrier
irish-electricity	idiot80	2 454 709	2 546 255	primal infeas. + barrier

The **irish-electricity** instance fails on *all four* methods, suggesting a problem-specific pathology (the instance triggers the barrier solver internally, which then fails to converge to a primal-feasible point). The **gsvm2r15** failure is specific to **dualpesteeep** only.

3.4 Instances with No Solution (NA / Killed)

Of the 58 instances that produced at least one NA or kill across any method:

- **9 instances** timed out on all four methods — these are genuinely hard LP relaxations within the per-instance time budget.
- **49 instances** failed on only some methods, indicating method-specific difficulty. **idiot30** is disproportionately represented in the method-specific failures.

4 Performance Analysis

4.1 Summary Statistics

Table 4 reports timing statistics over the 1 257 instances where all four methods solved correctly within the time limit.

Table 4: Timing statistics on 1 257 commonly solved instances (seconds).

Method	Median	Mean	Geom. mean	Max
idiot30	6.70	219.7	7.27	8238.6
idiot60	7.56	190.2	7.41	8915.9
idiot80	8.33	177.6	7.91	7226.4
dualpesteepest	2.99	287.7	5.07	8747.6

`dualpesteepest` has the lowest median and geometric mean times, but the highest arithmetic mean — a consequence of catastrophic slowdowns on a subset of instances (see Section 4.7).

4.2 Fastest-Method Count and Rank Distribution

Table 5: How often each method ranks 1st (fastest) and last (slowest) across 1 257 commonly solved instances.

Method	Rank 1 (fastest)	Rank 2	Rank 3	Rank 4 (slowest)
idiot30	156 (12.4%)	537	294	263
idiot60	176 (14.0%)	360	490	225
idiot80	152 (12.1%)	249	384	466 (37.1%)
dualpesteepest	770 (61.3%)	102	80	300

`dualpesteepest` is the fastest method on 61% of instances. `idiot80` is the slowest most often (37% of instances), making it the weakest of the four configurations.

4.3 Performance Profile

Table 6 shows the fraction of instances each method solves within a factor τ of the best (fastest) method on that instance — the standard performance profile metric [1].

Table 6: Performance profile: fraction of instances within $\tau \times$ best time.

Method	$\tau=1.0$	$\tau=1.25$	$\tau=1.5$	$\tau=2$	$\tau=3$	$\tau=5$	$\tau=10$
idiot30	0.126	0.301	0.446	0.607	0.741	0.839	0.924
idiot60	0.140	0.350	0.474	0.611	0.738	0.847	0.918
idiot80	0.122	0.305	0.434	0.580	0.712	0.826	0.913
dualpesteepest	0.615	0.713	0.758	0.796	0.850	0.908	0.960

`dualpesteepest` dominates the performance profile at every threshold. At $\tau = 1$ it is within the best time on 61.5% of instances (i.e. it is the best method). At $\tau = 10$ it is still within $10 \times$ best on 96% of instances, versus 91–92% for the idiot variants.

4.4 Geometric Mean Ratio vs Virtual Best

A complementary measure is the geometric mean of the ratio $t_{\text{method}}/t_{\text{best}}$ over all instances:

Method	Geom. mean ratio
idiot30	2.176
idiot60	2.142
idiot80	2.304
dualpesteeep	1.466

On average, **dualpesteeep** is $1.47\times$ the best method, while the idiot variants are $2.1\text{--}2.3\times$ the best.

4.5 Pairwise Comparison

Table 7 reports pairwise median runtime ratios. A ratio < 1 means the row method is faster than the column method on the median instance.

Table 7: Pairwise median runtime ratio (row / column). Values < 1 indicate the row method is faster.

	idiot30	idiot60	idiot80	dualpesteeep
idiot30	—	0.944	0.898	2.244
idiot60	1.059	—	0.929	2.393
idiot80	1.114	1.076	—	2.627
dualpesteeep	0.446	0.418	0.381	—

Among the idiot methods, **idiot30** is fastest overall (median ratio < 1 vs both **idiot60** and **idiot80**). The differences between the three idiot variants are modest (5–11% on the median), while **dualpesteeep** is roughly $2.2\text{--}2.6\times$ faster than any idiot variant on the median instance.

4.6 Significant Wins (Method-Specific Advantage)

A method has a “significant win” on an instance if it is the fastest and more than $2\times$ faster than the next best method. This identifies instances where a particular method provides a qualitative advantage.

Method	Significant wins ($> 2\times$ faster)
idiot30	10
idiot60	0
idiot80	0
dualpesteeep	365

- **dualpesteeep** dominates: 365 instances with $> 2\times$ advantage, including extreme cases up to $54\times$ faster (Table 8).
- **idiot30** has 10 specific wins, concentrated on bin-packing and set-covering type instances (e.g. **bppc6-02**, **bppc6-06**, **c1_06_020_***).
- **idiot60** and **idiot80** have zero significant wins. Neither method is ever more than $2\times$ faster than all alternatives on any instance. This strongly suggests that at least one (and possibly both) of these configurations could be removed from the **guess()** selection pool without measurable loss.

4.7 Extreme Cases

Table 8 shows the five instances where **dualpesteeep** is most advantageous and most disadvantageous relative to the best idiot method.

Table 8: Extreme performance cases for **dualpesteeep**.

Instance	Type	dualpesteeep	Best idiot	Ratio
neos-5129192-manaia	win	17.2s	920.8s	53.6× faster
neos-3755335-nizao	win	4.0s	134.4s	33.5× faster
pb-fit2d	win	28.6s	921.6s	32.2× faster
A-2	win	0.7s	21.9s	30.0× faster
neos-957270	win	0.8s	23.7s	28.6× faster
j12016_10	loss	6196.4s	79.4s	78.0× slower
supportcase42	loss	7270.5s	93.8s	77.5× slower
scpm1	loss	6791.4s	191.2s	35.5× slower
neos-933966	loss	326.8s	9.7s	33.7× slower
neos-935769	loss	173.0s	5.5s	31.6× slower

The catastrophic losses of **dualpesteeep** suggest that the **guess()** branching condition (based on **allInt** and coefficient statistics) sometimes selects this configuration for instances where it performs extremely poorly.

5 Conclusions and Recommendations

5.1 Memory Safety

All four configurations are memory-safe: **0 ASan errors**, **0 assertion failures** across 5 268 runs.

5.2 Correctness

- **irish-electricity** fails on all four methods due to an internal barrier solver convergence issue — this is an instance-level pathology, not configuration-specific.
- **gsvm2r15** fails on **dualpesteeep** only (**Unscaled problem has primal infeasibilities**). The other three methods solve it correctly. This is a mild correctness concern for the **dualpesteeep** path.

5.3 Reliability

dualpesteeep has the fewest kills (30 vs. 32–40 for idiot methods) and the lowest overall problem rate (2.4%). **idiot30** is the least reliable.

5.4 Performance

- **dualpesteeep** is by far the strongest method: fastest on 61% of instances, geometric mean ratio 1.47× best vs. 2.1–2.3× for idiot methods, and 365 significant wins ($> 2\times$ advantage).
- Among idiot methods, **idiot30** is the best overall (lowest geometric mean, highest rank-1 count among the three), with 10 specific wins on set-covering/bin-packing instances.

- **idiot60 and idiot80 have no significant wins and are nearly indistinguishable from each other.** Removing one or both from the `guess()` portfolio would not sacrifice measurable performance on any instance class tested.

5.5 Recommended Action

1. Investigate the `gsvm2r15` / `dualpesteeep` infeasibility issue — it may be a scaling or perturbation artefact specific to the `pesteeep` pivot rule.
2. Consider replacing `idiot60`/`idiot80` in `guess()` with `idiot30`, retaining only two configurations (`idiot30 + dualpesteeep`) differentiated by the `allInt` flag.
3. The 9 instances that time out on all four methods may benefit from a fallback to the standard dual simplex (`initialSolve`).

References

- [1] E. D. Dolan and J. J. Moré, *Benchmarking optimization software with performance profiles*, Mathematical Programming, 91(2):201–213, 2002.